

单调栈与单调队列在蓝桥杯中出现较少，大家只需要掌握基本的模板即可。

单调栈

1. 基本思想

单调栈求解的基本问题

在一个线性数据结构中，为任意一个元素找左边和右边第一个比自己大/小的位置，要求 $O(n)$ 的复杂度

对于这类问题，基本解法很容易想到 $O(n^2)$ 的解法，不过这种解法明显太基础了，有优化的空间。单调栈就可以优化基本解法的时间复杂度。

时间复杂度

单调栈是 $O(n)$ 的。

时间复杂度的证明：所有的元素只会进栈一次，而且一旦出栈后就不会再进来了。因此总的进出次数最多是 $2n$ 次，因此时间复杂度为 $O(n)$ 。

单调栈的性质

单调栈里的元素具有单调性：

1. 单调递减栈=>向栈生长的地方单调递减；
2. 单调递增栈=>向栈生长的地方单调递增

元素加入栈前，会在栈顶端把破坏栈单调性的元素都删除！

使用单调栈可以找到元素向左遍历第一个比他小的元素，也可以找到元素向左遍历第一个比他大的元素。

一般解题思路

1. 栈中不存放值，而是存放下标；
2. 在尝试去将一个元素加入栈前，先对它和栈顶元素进行比较，并按照单调性要求，把破坏栈单调性的栈顶元素都删除，直至满足单调性。

```
#include <bits/stdc++.h>
using namespace std;

int n, x;
int stk[100005], top;
int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++)
    {
        cin >> x;
        while(top > 0 && stk[top] >= x) top--;
        if(top > 0) cout << stk[top] << ' ';
        else cout << "-1 ";
        stk[++top] = x;
    }
}
```

```
}  
    return 0;  
}
```

单调队列

问题引入

给定一个长度为 n ($n \leq 10^7$) 的数组 $A[]$ 和长度为 k ($k \leq 10^5$) 的窗口，这个窗口从数组左端向右端滑动，求每向前滑动一次后，窗口内元素的最小值，并输出它们。例如：

$n = 8$	1	3	-1	-3	5	3	6	7
$k = 3$								



从暴力枚举到单调队列

```
for(int i = 1; i <= n-k+1; i++)  
{  
    int minn = INT_MAX;  
    for(int j = i; j < i+k; j++)  
        minn = min(minn, a[j]);  
    cout << minn << " ";  
}
```

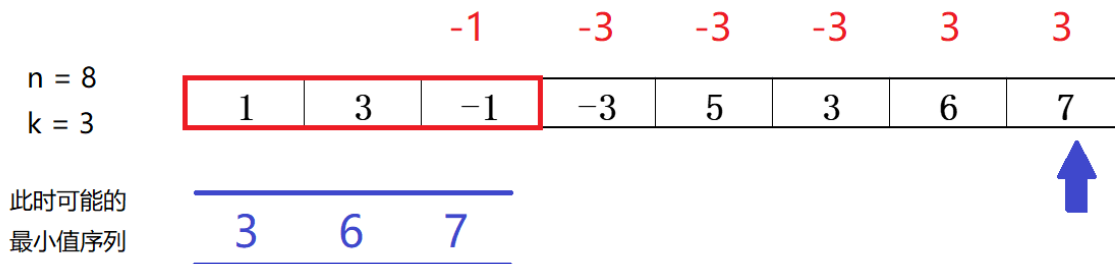
时间复杂度 $O(nk)$ ，太高，如何优化？

考虑尽可能排除掉不可能是答案的值，第一层循环不能降低了，只能考虑降低第二层循环的次数。比如下图中，当处理到 -1 的时候，很明显，前面的 1 和 3 不会成为当前及之后的窗口中的最小值（因为 -1 比它们更靠后），可以直接排除掉了。

$n = 8$	1	3	-1	-3	5	3	6	7
$k = 3$								

更一般地，如果 $A[i]$ 和 $A[j]$ 处在同一个窗口内，并且 $i < j$ 且 $A[i] \geq A[j]$ ，那么，必有 $A[i]$ 不可能是当前窗口内的最小值，也不可能是后续窗口内的最小值。即 $A[i]$ 是一个无用的数。

总结一下，我们发现，当处理 $A[j]$ 时， $A[j]$ 之前的所有比 $A[j]$ 大的元素都可以排除，不用考虑。由此，我们可以得出上面例子中的各个滑动窗口内的最小值如下：



最后，当滑动到 7 时，此时可能的解有三个：3,6,7。元素 -1 在处理到元素 3 时被排除掉，元素 -3 在处理元素 6 时被排除掉。

对于上面这种既有序，又单调的序列，我们称之为“单调队列”。

性质

单调队列中元素之间的关系具有单调性，分为 **单调递增队列** 和 **单调递减队列**。

1. 队列中的元素在原来的列表中的位置是由前往后的(随着循环顺序入队)。
2. 队列中的队首元素和队尾元素的距离应不超过问题中规定的区间长度（**区间性**）
3. 队列中元素的大小是单调递增或递减的（**单调性**）。

因此，

1. 为了保证队列的区间性，那么我们在将元素入队之前，应该将队列中会破坏区间性的（队首）元素删除掉。
2. 为了保证队列的单调性，那么我们在将元素入队之前，应该将队列中会破坏单调性的（队尾）元素删除掉。

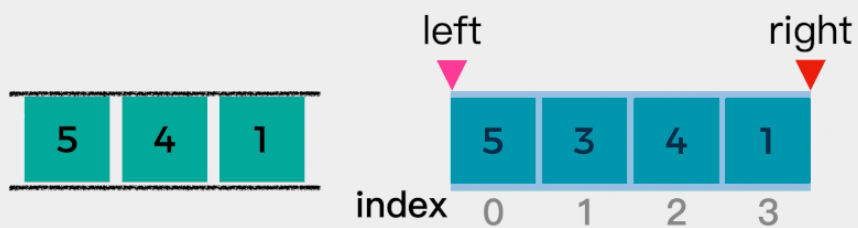
操作

单调队列和单调栈在实现上有一点是相同的：**当需要将一个元素加入队列或栈时，就将破坏队列或栈的单调性的元素删除**。区别是单调栈只在栈顶进出元素，而单调队列为了维持单调性，需要**同时在队首和队尾弹出元素**。因此，单调队列实质上是一类 **双端队列（deque）**。

单调队列在加入一个元素时，我们通过一定操作维护队列的单调性。该队列特征的操作便是“后移一位”。顾名思义，后移一位就是指队列维护的区间往后移一位。这个操作中要做的事情是弹出队首与加入队尾。

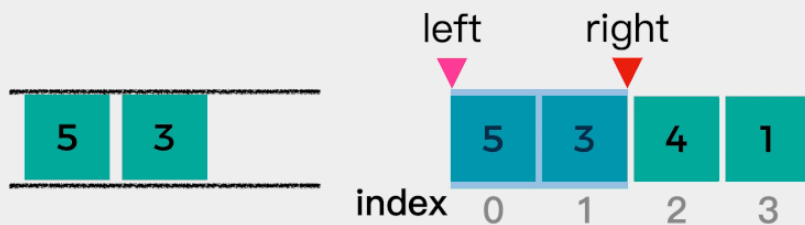
1. 弹出队首

对于插入的元素，我们检查它与队首元素的位置之差，当这个 **差大于队列定长时说明队首元素已在区间之外**，应当弹出。



2. 加入队尾

为了保证这个队列中元素的单调性，对于加入的元素，我们可能要删除一些队尾的元素。以单调递增队列为例，插入时要 **弹出所有比要插入元素大的队尾元素，直到队尾元素不再比其大再插入**。此时插入元素时要记下插入元素在原序列中的位置，这个信息将会对弹出队首时发挥作用。



代码实现

1. 手写队列

```

int q[N], head = 0, tail = 0;
for (int i = 0; i < n; i++) {
    while (head < tail && check_out(q[head], i)) head++; // 队头滑出窗口区域就弹出队首
    while (head < tail && check(q[tail-1], i)) tail--; // 队尾不满足单调性就弹出队尾
    q[tail++] = i; // 当前元素入队
}

```

2. STL 双端队列 deque

deque (double-ended queue, 双端队列) 是一种具有队列和栈的性质的数据结构。双端队列中的元素可以从两端弹出，并且相比 list 增加了 [] 运算符重载。

```

#include <deque> // 头文件
deque<int> dq; // 定义
dq.push_back(t); // 在队尾插入元素 t
dq.push_front(t); // 在队首插入元素 t
dq.pop_back(); // 删除队尾元素
dq.pop_front(); // 删除队首元素
dq.front() = t; // 获取队首元素的引用，并修改为 t
dq.back() = t; // 获取队尾元素的引用，并修改为 t
dq[pos] = t; // 获取下标元素的引用，并修改为 t
int len = dq.size(); // 获取队列大小
dq.clear(); // 清空队列
bool flag = dq.empty(); // 判空

// 遍历输出队列
for(int i = 0; i < dq.size(); i++)
    cout << dq[i] << ' ';

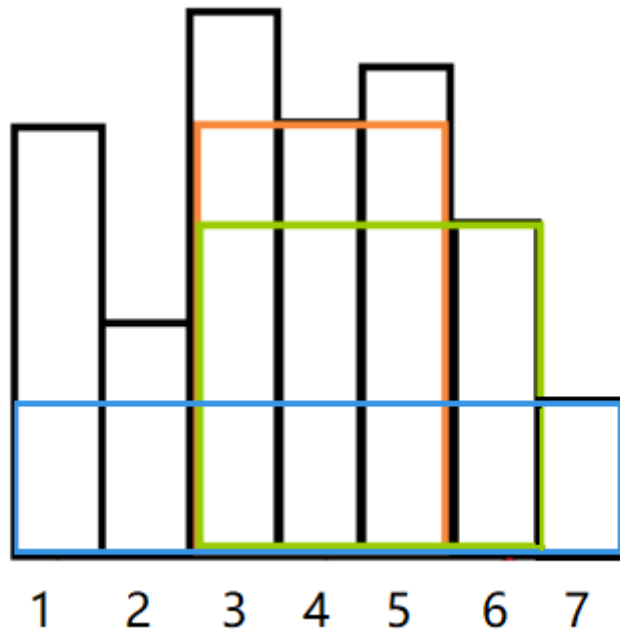
// 双端队列实现单调递增队列
deque<int> q; // 队列保存下标即可
for(int i = 1; i <= n; i++) {
    while(!q.empty() && q.front() < i-len+1) q.pop_front(); // 队首元素已经出了长为 len 的窗口，需要出队
    while(!q.empty() && a[q.back()] >= a[i]) q.pop_back(); // 队列不空，且队尾元素大于当前元素，不满足单调递增
    q.push_back(i); // 当前元素（下标）入队
}

```

蓝桥杯真题

直方图中最大的矩形

依旧记录每个矩形可向两侧扩展的左右边界。但是在计算每个矩形可以扩展的左/右边界时，可以发现我们真正关心的是第一个比当前矮的矩形，因为它决定了可以左/右扩展的长度。而这类“求左边和右边第一个比自己大/小的位置”的问题，单调栈可以解决。并且时间复杂度是 $O(n)$ 的。



如上图所示，每个矩形高度从左向右依次是：5,3,7,5,6,4,2（代码中左右补两个高度 -1 的矩形）在求每个矩形向左扩展时的最大长度时，我们维护一个单调栈即可。因为要找的是左侧第一个比当前高度矮的矩形，因此是一个单调递增栈。

矩形编号	当前矩形高度	当前的单调栈
0	-1	[]
1	5	[0]，向左长度为 1-0=1，向左最大面积为 5
2	3	[0]，向左长度为 2-0=2，向左最大面积为 6
3	7	[0,2]，向左长度为 3-2=1，向左最大面积为 7
4	5	[0,2]，向左长度为 4-2=2，向左最大面积为 10
5	6	[0,2,4]，向左长度为 5-4=1，向左最大面积为 6
6	4	[0,2]，向左长度为 6-2=4，向左最大面积为 16
7	2	[0]，向左长度为 7-0=7，向左最大面积为 14

定义单调递增栈去统计每个矩形右侧第一个小于当前矩形的矩形的位置（求出向右扩展长度）。然后再定义一个数组保存向左能扩展到的大于当前高度的长度（当不满足单调栈特性时，求和就可以求出向左扩展的最大长度）。如此一来，对于栈顶元素的向左右扩展的最大面积就能求出来了。

```
#include<iostream>
#include<algorithm>

using namespace std;

const int N = 100010;

//l[i]，r[i]表示第i个矩形的高度可向两侧扩展的左右边界
int h[N], q[N], l[N], r[N];

typedef long long LL;
```

```

int main()
{
    int n;
    while(scanf("%d", &n), n)
    {
        for(int i = 1; i <= n; i++) scanf("%d", &h[i]);
        h[0] = h[n + 1] = -1; // 为了方便处理开头和结尾，左右加两个高度为 -1 的矩形，不
影响结果

        // 处理向左的最大扩展长度
        int top = -1;
        q[++ top] = 0; // q[top]: 存小于当前位置高度的第一个矩形的下标
        for(int i = 1; i <= n; i++)
        {
            while(h[q[top]] >= h[i]) top--;
            l[i] = i - q[top]; // l[i]: 存 i 位置向左的最大扩展长度
            q[++ top] = i;
        }
        // 处理向右的最大扩展长度
        top = -1;
        q[++ top] = n + 1;
        for(int i = n; i; i--)
        {
            while(h[q[top]] >= h[i]) top--;
            r[i] = q[top] - i; // r[i]: 存 i 位置向右的最大扩展长度
            q[++ top] = i;
        }
        // 寻找最大面积
        LL res = 0;
        for(int i = 1; i <= n; i++) res = max(res, h[i] * (l[i] + r[i] -
1));
        printf("%lld\n", res);
    }
    return 0;
}

```

习题

小蓝与赛马娘

这道题思路简单几乎是一个单调队列的板子题

“当一头马娘左边 D 距离内和右边 D 距离内都有身高至少是它的两倍的马娘，就会觉得不自在影响她的比赛发挥。”

这句话提示它是一个滑动窗口，窗口长度是 D ，身高至少是两倍，那么只要窗口内的最大值大于等于当前高度两倍就可以判断为从某一侧拥挤了。

于是，只需要从左到右和从右到左都分别跑一遍单调队列滑动窗口，去判断每侧是否会拥挤，如果都拥挤，那么就答案加一。

```

#include <bits/stdc++.h>
using namespace std;

const int N = 5e4+5;
int n, d, x[N], h[N];
pair<int, int> a[N];

```

```

bool c1[N], cr[N];
int main(){
    cin >> n >> d;
    for(int i = 1; i <= n; i++) {
        cin >> a[i].first >> a[i].second;
    }
    sort(a+1, a+n+1);
    for(int i = 1; i <= n; i++) {
        x[i] = a[i].first;
        h[i] = a[i].second;
    }
    deque<int> q;
    for(int i = 1; i <= n; i++) {
        while(!q.empty() && h[q.back()] < h[i]) q.pop_back();
        while(!q.empty() && x[i]-x[q.front()]>d) q.pop_front();
        if(!q.empty() && h[q.front()] >= 2*h[i]) c1[i] = true;
        q.push_back(i);
    }
    q.clear();
    for(int i = n; i >= 1; i--) {
        while(!q.empty() && h[q.back()] < h[i]) q.pop_back();
        while(!q.empty() && x[q.front()]-x[i]>d) q.pop_front();
        if(!q.empty() && h[q.front()] >= 2*h[i]) cr[i] = true;
        q.push_back(i);
    }
    int ans = 0;
    for(int i = 1; i <= n; i++) ans += (c1[i]&&cr[i]);
    cout << ans;
    return 0;
}

```

小蓝与排队

仔细理解题意，题目可以转化为找当前数字向右查找的第一个大于他的数字之间有多少个数字，然后将每个结果累计就是答案。重点在于找到每个数字右边第一个大于他的数字的下标，如果知道对应的下标，就可以计算出之间的数字个数（距离）。具体可以用单调栈来处理。

从前往后遍历的单调递减栈

核心思想：从前往后计算，统计每个人可以挡住几个人的视野，以及对应的被挡住的人能看到的数量。

题中说，个子高的可以看到个子矮的，所以我们可以维护一个单调递减栈。然后在维护过程中对于当前的人如果发现单调栈的栈顶的人身高比当前的矮，就意味着栈顶的人被当前的人挡住了，因此栈顶的人能看到的人数也就确定了，如此处理即可。需要注意的是，可以在最后一个人后面插入一个极高的人，以方便处理没人能挡住视野的情况。具体见代码：

```

#include<iostream>
#include<climits>

using namespace std;

const int N = 100010;
long long h[N];
long long stk[N], top; // 一个单调递减栈
long long ans;
int main()

```



```

{
    int n,t;
    scanf("%d", &n);
    for(int i = 1; i <= n; i++) scanf("%d", &h[i]);
    h[n+1] = INT_MAX;
    for(int i = 1; i<=n+1; i++)
    {
        // 只要编号 i 能挡住 stk[top] 的视野，就计算 stk[top] 能看到的人
        while(top && h[stk[top]] <= h[i])
        {
            ans += i - stk[top] - 1; // stk[top] 能看到的人数
            top--; // stk[top] 出栈，不再参与计算
        }
        stk[++top] = i;
    }
    cout << ans;
    return 0;
}

```

```

#include <bits/stdc++.h>
using namespace std;

long long n, h[100005], ans;
stack<long long> s;

int main()
{
    cin >> n;
    for(int i = 1; i <= n+1; i++)
    {
        if(i < n+1) cin >> h[i];
        else h[i] = INT_MAX;
        while(!s.empty() && h[s.top()] <= h[i]) // s.top() 比 i 矮
        {
            // s.top() 能看到 i 前面一个人
            ans += i - s.top() - 1;
            s.pop(); // s 统计完了，出去
        }
        s.push(i);
    }
    cout << ans;
    return 0;
}

```